Plano de Garantia da Qualidade de Software (PGQ)

Padrões, revisões, testes, métricas, gestão de defeitos e riscos do projeto

Campo	Informação
Projeto	Sistema de Folha de Pagamento — Mercadinho Fapa
Disciplina	Garantia da Qualidade de Software — Trabalho Integrador (A1 e A3)
Equipe	Guilherme Belotto, Kauã e Leonardo Trinca
Data	20/10/2026
Versão	1.0

Histórico de revisões

Versão	Data	Descrição	Responsáveis
1.0	20/10/2026	Versão inicial para a entrega A1.	Equipe

1. Objetivo e escopo

Este plano define como a equipe vai garantir a qualidade do Sistema de Folha de Pagamento do Mercadinho Fapa, dos documentos da A1 ao sistema entregue na A3. Vale para todos os artefatos do repositório: documentos, modelos, código, banco de dados, testes e manuais.

2. Documentos de referência

- Documento de Visão e ERS (01_Requisitos)
- Modelagem UML (02_Analise) e Documento de Arquitetura (03_Projeto)
- Plano de Testes (04_Testes)

3. Organização e responsabilidades

Papel	Responsável	Atribuições de qualidade
Coordenação e configuração	Kauã	Manter o repositório organizado, o cronograma e as versões dos documentos.
Liderança técnica	Guilherme	Garantir a arquitetura em camadas, as convenções de código e a revisão técnica.
Líder de qualidade e testes	Leo	Manter o Plano de Testes, o registro de defeitos, as métricas e o relatório final de qualidade.
Toda a equipe	Kauã, Guilherme e Leo	Revisão cruzada: ninguém aprova o próprio trabalho; testes de aceitação executados por quem não desenvolveu o módulo.

4. Padrões e convenções

4.1 Documentos

- Formato Word (.docx) com capa, histórico de revisões e identificação do projeto; PDF gerado na entrega.
- Versão v1.0 na entrega A1; alterações posteriores incrementam a versão e registram o motivo no histórico.
- Cada documento fica na pasta correspondente do repositório (ver README).

4.2 Código Java

Tema	Padrão
Nomes	Pacotes em minúsculas (`br.fapa.folha.service`); classes em PascalCase; métodos e variáveis em camelCase; constantes em MAIÚSCULAS.
Formatação	Indentação de 4 espaços; uma classe por arquivo; linhas de até ~120 caracteres.
Camadas	Sem regra de negócio no Controller; sem acesso a dados fora dos Repositories; Service não conhece HTML.
Dinheiro	`BigDecimal` com 2 casas (RN15, RNF04); nunca `double`.
Segurança	Senha sempre com BCrypt; colaborador consultado pelo funcionário vem do usuário autenticado, nunca da URL.
Comentários	Javadoc nos métodos públicos dos Services; comentar o “porquê”, não o óbvio.

4.3 Git e GitHub

Tema	Padrão
Branches	`main` (estável, só recebe merge por Pull Request); `develop` (integração); `feature/<modulo>-<descricao>` por tarefa.
Commits	Formato `tipo: descrição` com `feat`, `fix`, `docs`, `test`, `refactor`, `chore`. Exemplo: `feat: validar política de senha no cadastro`.
Pull Requests	Todo merge em `develop`/`main` precisa de revisão de pelo menos 1 colega (checklist 7.2).
Tags	`v1.0-A1` na entrega da A1 e `v1.0-A3` na entrega final.
Autoria	Cada integrante faz os commits do que produziu, para o histórico mostrar a contribuição real.

5. Atividades de garantia da qualidade

Atividade	O que é	Quando	Responsável
Revisão de documentos	Leitura cruzada com o checklist 7.1, conferindo consistência entre Visão, ERS, UML, Arquitetura e Testes.	Antes de cada entrega	Quem não escreveu o documento
Revisão técnica de código	Revisão em Pull Request com o checklist 7.2.	A cada Pull Request	Guilherme + colega
Testes unitários	JUnit 5 e Mockito na camada de serviço (regras RN, geração da folha, política de senha).	Junto com o desenvolvimento	Autor do módulo
Testes de integração	Controllers, Services e Repositories com banco de testes.	Após a integração	Leo
Testes de aceitação	Execução dos 40 casos do Plano de Testes pela interface.	Antes da homologação	Quem não desenvolveu o módulo
Testes de segurança	CT05, CT06, CT08 e CT09: acesso entre perfis e proteção de páginas.	A cada entrega de módulo	Leo
Gestão de defeitos	Registro, classificação, correção e reteste (seção 8).	Contínuo	Leo
Análise de métricas	Coleta e análise das métricas da seção 6.	Semanal e no relatório final	Leo
Homologação	Validação contra os critérios de aceitação da ERS, com termo.	Fim de novembro	Equipe

6. Métricas de qualidade

Métrica	Como calcular	Meta	Fonte
Cobertura de testes unitários (camada de serviço)	Linhas cobertas ÷ linhas totais	≥ 80%	Relatório JaCoCo
Taxa de aprovação dos casos de teste	CT aprovados ÷ CT executados	100% dos de prioridade Alta; ≥ 95% no total	Registro de execução
Rastreabilidade	RF com pelo menos 1 CT ÷ total de RF	100%	Matriz da ERS
Defeitos por requisito	Defeitos registrados ÷ nº de RF	Acompanhar (tendência decrescente)	Registro de defeitos
Defeitos críticos/altos abertos	Contagem na homologação	0	Registro de defeitos
Taxa de reabertura	Defeitos reabertos ÷ corrigidos	≤ 10%	Registro de defeitos
Desvio de cronograma	Data real − data planejada de cada marco	≤ 3 dias	Cronograma
Achados de revisão	Itens apontados nas revisões	Todos tratados antes da entrega	Comentários dos Pull Requests

7. Checklists de revisão

7.1 Documentos

- Capa com nome do projeto, equipe, data, versão e histórico de revisões.
- Códigos (RF, RN, UC, CA, CT) usados do mesmo jeito em todos os documentos.
- Todo RF tem caso de uso, regra associada, critério de aceitação e ao menos um caso de teste.
- Escopo coerente: funcionários consultam só o próprio; sem cálculo de impostos.
- Texto sem erros de português e sem trechos soltos ou duplicados.

7.2 Código (Pull Request)

- Compila e os testes existentes passam.
- Segue convenções de nomes e regras de camadas (4.2).
- Regra de negócio implementada conforme a RN citada.
- Validações com mensagem clara em português (RNF05).
- Rotas protegidas pelo perfil correto; nenhuma senha ou credencial no código.
- Testes unitários para a lógica nova da camada de serviço.
- Mensagem de commit no padrão.

8. Gestão de defeitos

8.1 Ciclo de vida

Novo → Em correção → Corrigido → Verificado (reteste aprovado) → Fechado. Se o reteste falhar, volta para Em correção e conta como reaberto.

8.2 Severidade

Severidade	Definição	Exemplo
Crítica	Impede usar o sistema ou expõe dados salariais de outra pessoa.	Funcionário consegue ver o histórico de outro.
Alta	Função principal falha, mas há contorno.	Folha duplicada é aceita.
Média	Função secundária falha ou mensagem confusa.	Filtro de consulta não funciona.
Baixa	Aparência ou texto sem efeito no resultado.	Erro de ortografia em tela.

8.3 Registro de defeitos

Mantido em `10_Qualidade/Registro_de_Defeitos.xlsx`, com as colunas abaixo, listas de seleção e uma aba de métricas.

ID	Data	CT / RF	Descrição	Severidade	Responsável	Status	Reteste
DEF-001	dd/mm	CT00	Passos e resultado obtido	Alta	Nome	Novo	—

9. Critérios de aceitação do produto (definição de “pronto”)

- Requisitos funcionais de prioridade Alta e Média implementados e demonstráveis.
- Casos de teste executados, sem defeitos críticos ou altos abertos.
- Cobertura de testes unitários da camada de serviço ≥ 80%.
- Manuais do usuário e técnico, termo de homologação e relatório final de qualidade no repositório.
- Repositório na estrutura de pastas exigida, com histórico de commits de todos os integrantes.

10. Gestão de riscos

Risco	Prob.	Impacto	Mitigação	Resp.
Atraso por pouca prática com Spring Boot e JPA	Alta	Alto	Começar pelo cadastro e login; pareamento entre os três; cortar RF13 (PDF) se necessário.	Guilherme
Falha de segurança entre perfis	Média	Alto	Spring Security com rotas por perfil; testes CT05, CT06, CT09; revisão de PR.	Guilherme
Dificuldade com instalação do PostgreSQL	Média	Médio	Guia de instalação no manual técnico; opção de usar contêiner Docker.	Leo
Conflitos de merge no Git	Média	Médio	Módulos separados por pacote; branches curtas; merges frequentes em `develop`.	Kauã
Mudança de requisito perto da entrega	Baixa	Alto	Mudanças só por decisão conjunta, com atualização da ERS e do Plano de Testes.	Kauã
Indisponibilidade de um integrante	Média	Alto	Revisão cruzada garante que todos conhecem os módulos; documentação atualizada.	Equipe

11. Registros e evidências

Na pasta `10_Qualidade/` ficam este PGQ, o registro de defeitos, o relatório de cobertura (JaCoCo), a EAP e o cronograma e o relatório final de qualidade. Em `04_Testes/` ficam o Plano de Testes e os resultados de execução, com capturas de tela como evidência.